Double-click (or enter) to edit

```
import pandas as pd

# Load dataset
df = pd.read_csv('Housing.csv')

# Display first 10 rows
print("First 10 Rows:")
display(df.head(10))

# Check dataset shape
print("Dataset Shape:", df.shape)
```

```
First 10 Rows:
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basem
0	13300000	7420	4	2	3	yes	no	
1	12250000	8960	4	4	4	yes	no	
2	12250000	9960	3	2	2	yes	no	
3	12215000	7500	4	2	2	yes	no	
4	11410000	7420	4	1	2	yes	yes	
5	10850000	7500	3	3	1	yes	no	
6	10150000	8580	4	3	4	yes	no	
7	10150000	16200	5	3	2	yes	no	
8	9870000	8100	4	1	2	yes	yes	
9	9800000	5750	3	2	4	yes	yes	

```
Dataset Shape: (545, 13)
```

```
0# Check missing values
print("Missing Values in Each Column:")
print(df.isnull().sum())

# Target column
print("\nTarget Column: price")

# Feature columns
print("\nFeature Columns:")
print(df.drop('price', axis=1).columns.tolist())
```

```
Missing Values in Each Column:
price               0
area                0
bedrooms            0
bathrooms           0
stories             0
mainroad            0
guestroom           0
basement            0
hotwaterheating     0
airconditioning     0
parking             0
prefarea            0
furnishingstatus    0
dtype: int64

Target Column: price

Feature Columns:
['area', 'bedrooms', 'bathrooms', 'stories', 'mainroad', 'guestroom', 'ba
```

```
import pandas as pd

# Reload dataset
df = pd.read_csv('Housing.csv')

# Remove duplicate rows
df = df.drop_duplicates()

# Convert yes/no columns manually
df['mainroad'] = df['mainroad'].map({'yes': 1, 'no': 0})
df['guestroom'] = df['guestroom'].map({'yes': 1, 'no': 0})
df['basement'] = df['basement'].map({'yes': 1, 'no': 0})
df['hotwaterheating'] = df['hotwaterheating'].map({'yes': 1, 'no': 0})
df['airconditioning'] = df['airconditioning'].map({'yes': 1, 'no': 0})
df['prefarea'] = df['prefarea'].map({'yes': 1, 'no': 0})

# Convert furnishing status to numeric columns
df = pd.get_dummies(df,
                    columns=['furnishingstatus'],
                                        drop_first=True)
print("Dataset Shape:", df.shape)
df.head()
```

```
Dataset Shape: (545, 14)
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	baseme
0	13300000	7420	4	2	3	1	0	
1	12250000	8960	4	4	4	1	0	
2	12250000	9960	3	2	2	1	0	
3	12215000	7500	4	2	2	1	0	
4	11410000	7420	4	1	2	1	1	

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_
import numpy as np

# Separate features and target
X = df.drop('price', axis=1)
y = df['price']

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
    )

    # Create and train model
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

    # Make predictions
y_pred_lr = lr_model.predict(X_test)

    # Evaluate modelprint("Linear Regression Results")
print("MAE:", mean_absolute_error(y_test, y_pred_lr))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lr)))
print("R2 Score:", r2_score(y_test, y_pred_lr))
```

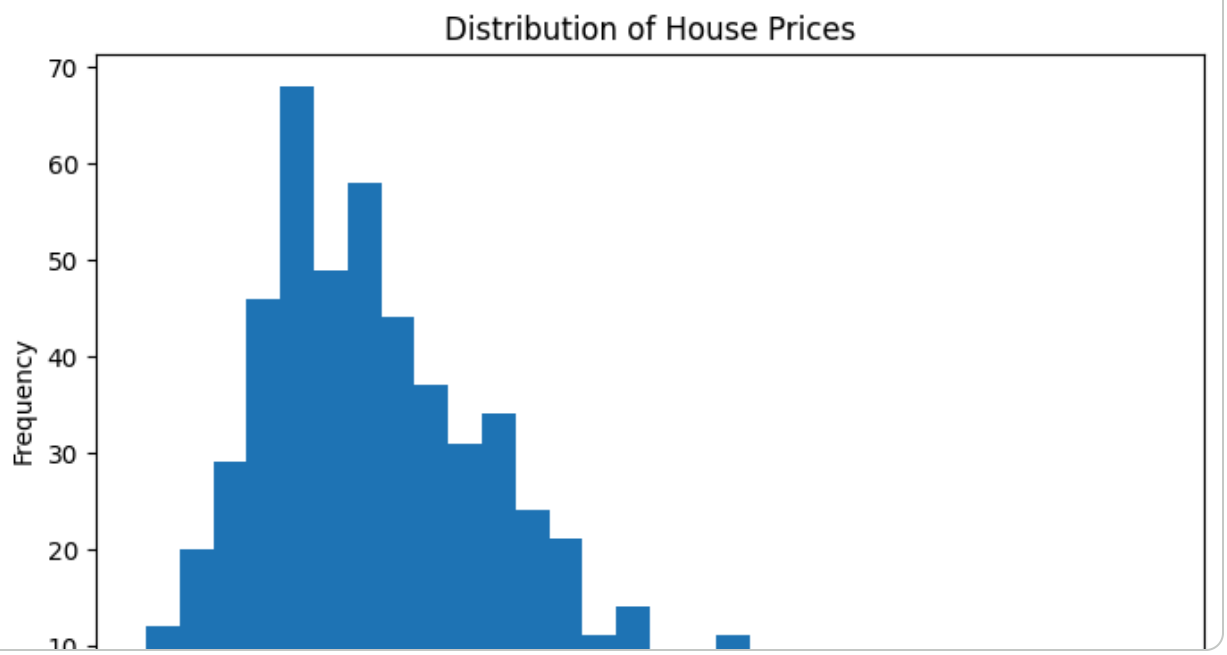
```
MAE: 970043.4039201637
RMSE: 1324506.9600914384
R2 Score: 0.6529242642153185
```

```
from sklearn.ensemble import RandomForestRegressor

# Create Random Forest model
rf_model = RandomForestRegressor(
    n_estimators=100,
        random_state=42
```

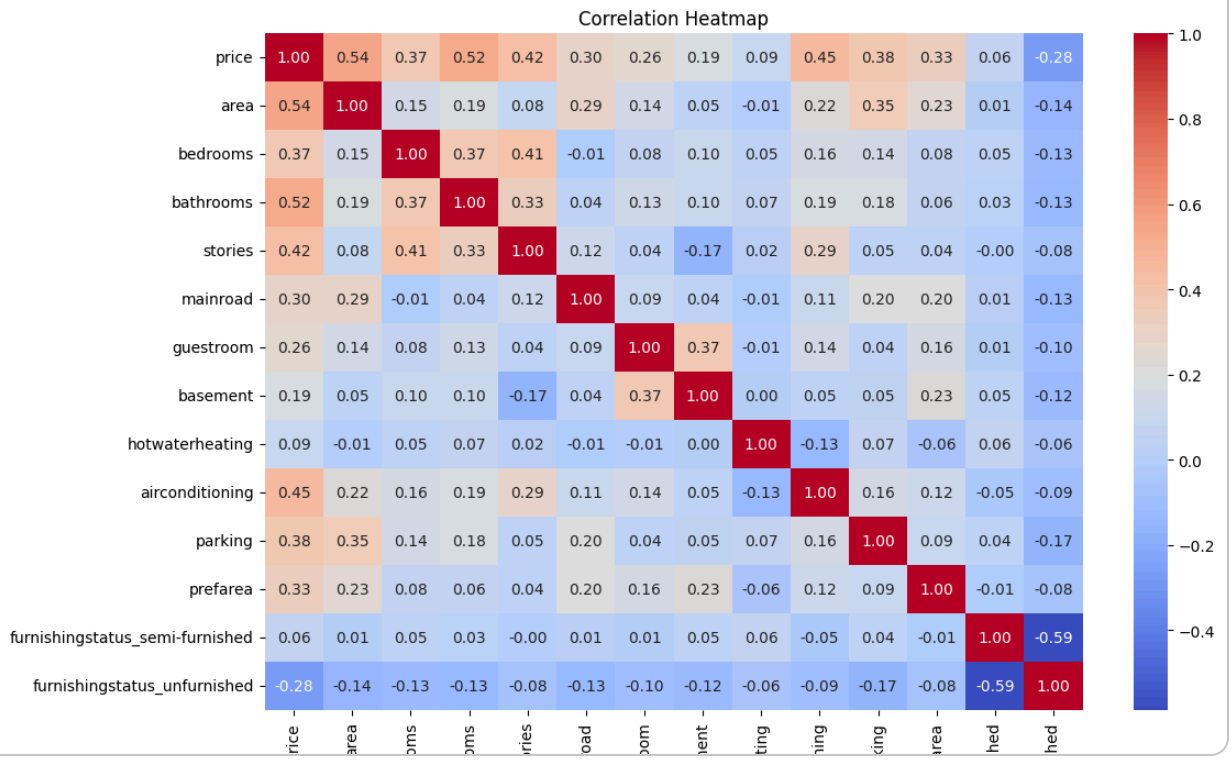
```
        )

        # Train model
rf_model.fit(X_train, y_train)

        # Make predictions
y_pred_rf = rf_model.predict(X_test)

        # Evaluate model
print("Random Forest Results")
print("MAE:", mean_absolute_error(y_test, y_pred_rf))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_rf)))
print("R2 Score:", r2_score(y_test, y_pred_rf))
```

```
Random Forest Results
MAE: 1022560.0527522935
RMSE: 1401496.8425384816
R2 Score: 0.6114024924156645
```

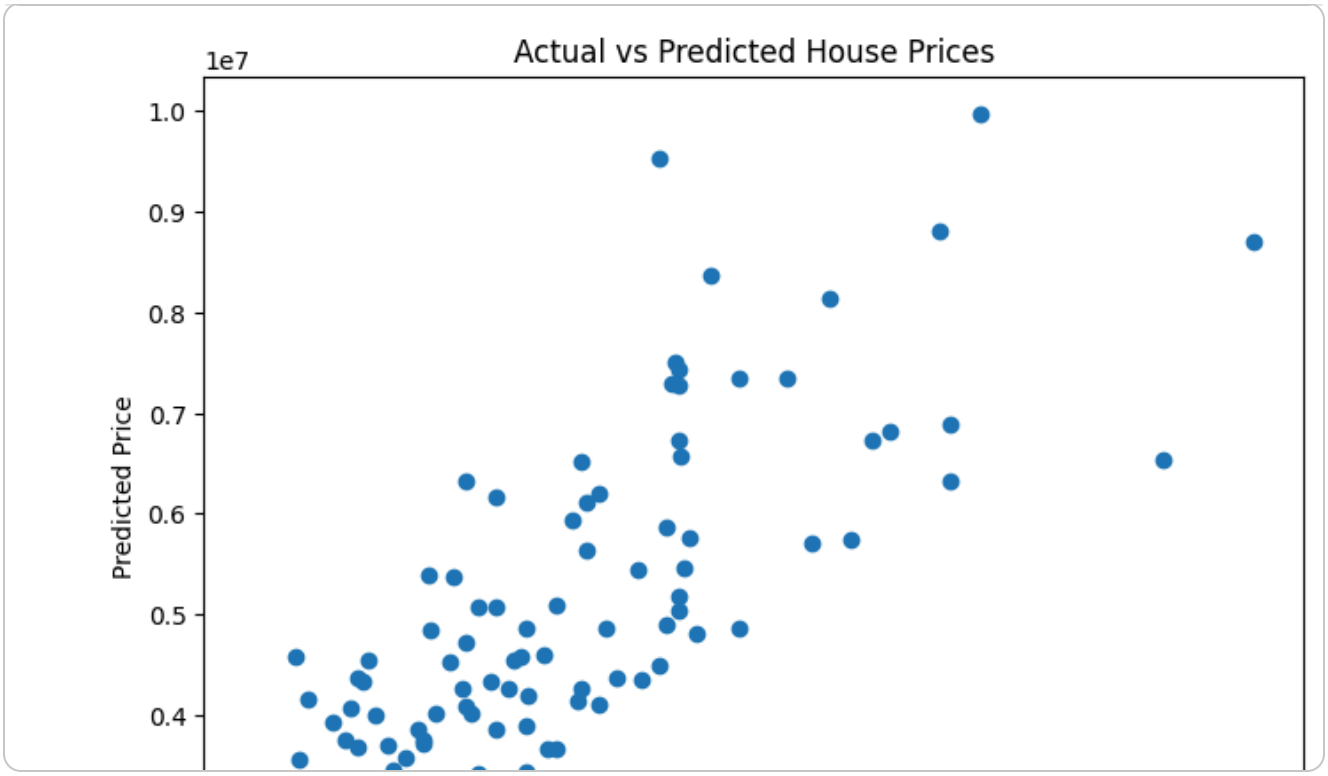
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.hist(df['price'], bins=30)

plt.title('Distribution of House Prices')
plt.xlabel('House Price')
plt.ylabel('Frequency')

plt.savefig('price_distribution.png')

plt.show()
```



```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12,8))

sns.heatmap(df.corr(),
            annot=True,
                        cmap='coolwarm',
                                    fmt='.2f')

plt.title('Correlation Heatmap')

plt.savefig('correlation_heatmap.png')

plt.show()
```




```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

plt.scatter(y_test, y_pred_rf)

plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted House Prices')

plt.savefig('actual_vs_predicted.png')

plt.show()
```



Insights and Summary

Based on the analysis, the area of the house, number of bathrooms, number of stories, and availability of air conditioning were among the most influential factors affecting house prices. The Random Forest Regressor performed better than the Linear Regression model, achieving an R^2 score of 0.6114, which indicates that the model was able to explain around 61% of the variation in house prices.

The analysis revealed that larger houses with more amenities generally had higher prices. One interesting observation was that houses located in preferred areas and having additional facilities such as air conditioning tended to be significantly more expensive.

Overall, the Random Forest model provided reasonably accurate predictions and outperformed Linear Regression. A recommendation for real estate businesses is to focus on properties with premium features and desirable locations, as these factors have a strong influence on house prices.